Project Report

# Powered Shopping

## AI-Driven Voice Shopping Assistant

AI-Driven Voice Shopping Assistant
Project Report
Generated on: 2026-04-21
Project Type: Full-stack web application
Technology Base: React, Vite, Express, Firebase, OpenAI

## 1. Abstract

Powered Shopping is a full-stack shopping assistant designed to make product discovery and cart management more natural through voice and text interaction. The project combines a React and Vite frontend with an Express backend, Firebase authentication, OpenAI-powered assistant capabilities, and multiple fallback mechanisms to keep the system usable even when external services are unavailable. The application supports secure login, voice search, typed commands, recommendations, cart operations, and checkout simulation in a single workflow.

The report presents the project problem, objectives, architecture, technology choices, module design, user flow, resilience strategy, challenges, and future scope. The implementation focuses not only on AI integration, but also on reliability, usability, and demo-readiness.

## 2. Problem Statement

Online shopping platforms usually require repeated manual interaction such as typing keywords, applying filters, browsing multiple pages, and separately managing the cart and checkout flow. For many users, this experience is time-consuming and not very conversational.

The goal of this project is to create a smarter shopping interface where a user can say or type commands such as:

- show Nike shoes under 2000
- add first item to cart
- recommend trending gadgets
- checkout now

The system should understand the request, perform the correct shopping action, and update the interface in real time.

## 3. Objectives

The main objectives of Powered Shopping are:

- build an e-commerce assistant with voice-first interaction
- allow users to search, filter, add, remove, and checkout using natural language
- integrate a Generative AI API for assistant reasoning and transcription
- provide secure user access through Firebase authentication
- ensure the application remains usable through fallback logic when AI or external services fail
- create a working prototype suitable for classroom demonstration and technical evaluation

## 4. Project Overview

Powered Shopping is organized as a client-server application.

The frontend is responsible for:

- authentication flow
- dashboard rendering
- product browsing
- voice and text command entry
- cart and checkout screens
- local UI preference persistence

The backend is responsible for:

- product retrieval and filtering
- recommendation generation
- cart operations
- AI assistant tool execution
- voice transcription handling

The project is designed around a practical user journey:

1. User signs in.
2. User searches products through voice or text.
3. User receives product results or recommendations.
4. User adds products to the cart.
5. User reviews the cart and completes checkout.

## 5. Technology Stack

### 5.1 Frontend

- React 18
- Vite 5
- CSS-based custom UI
- Web Speech API for browser speech recognition and speech synthesis

Reason for use:

- React provides component-based development and state-driven UI updates.
- Vite gives fast local development and lightweight bundling.
- Web Speech API allows live voice interaction directly in the browser.

### 5.2 Backend

- Node.js

- Express 4

Reason for use:

- Express keeps API routing simple and modular.
- Node.js fits well for JavaScript-based full-stack development and async API handling.

### 5.3 Authentication and Persistence

- Firebase Authentication
- Firebase Admin SDK
- Firestore as optional cart persistence

Reason for use:

- Firebase Authentication makes secure email/password and Google sign-in easier to integrate.
- Firestore enables cloud cart persistence when configured.
- If Firebase is unavailable, the system can still continue with local in-memory fallback logic.

### 5.4 AI and Voice Services

- OpenAI Chat Completions API
- OpenAI Whisper transcription API

Reason for use:

- Chat Completions enables assistant behavior with tool-calling.
- Whisper transcription handles recorded voice fallback when browser speech recognition is not reliable.

### 5.5 Product Data

- FakeStore API
- Local mock product JSON dataset

Reason for use:

- FakeStore API provides remote catalog data quickly for prototyping.
- Local mock data ensures the app remains functional when the remote API is slow or unavailable.

## 6. Key Features

Powered Shopping includes the following major features:

- secure sign-in using email/password and Google
- voice-based command input
- manual typed command input

- live spoken assistant replies
- product search by keyword, category, brand, price, and rating
- recommendation support for trending and similar items
- add to cart and remove from cart operations
- guided checkout simulation
- conversation history tracking
- multiple fallback layers for better reliability

## 7. System Architecture

The system is divided into four major layers:

### 7.1 Client Layer

The client is built with React and manages:

- authenticated entry into the app
- navigation between views
- voice control panel
- product listing and filters
- cart and checkout interfaces
- user preferences such as theme and assistant settings

Primary frontend files include:

- ‘client/src/main.jsx‘
- ‘client/src/pages/HomePage.jsx‘
- ‘client/src/components/VoiceControl.jsx‘
- ‘client/src/hooks/useFirebaseAuth.js‘
- ‘client/src/hooks/useSpeechRecognition.js‘

### 7.2 Backend API Layer

The backend exposes API routes under ‘/api‘ for:

- products
- categories
- recommendations
- cart
- AI chat
- AI transcription

Primary backend files include:

- ‘server/index.js‘
- ‘server/routes/*.js‘
- ‘server/controllers/*.js‘

- ‘server/services/*.js‘

### 7.3 AI Layer

The AI assistant service receives a user message and may call server-side tools for:

- searching products
- getting recommendations
- adding to cart
- removing from cart
- reading cart state
- checking out

This tool-calling approach allows the assistant to perform real actions instead of only returning text.

### 7.4 Data and Storage Layer

Data enters the system from:

- FakeStore API
- local mock product JSON
- Firestore, when available
- browser localStorage for lightweight preferences

## 8. Functional Modules

### 8.1 Authentication Module

The app is protected by Firebase Auth. Users must sign in before accessing the main dashboard. Supported methods include:

- email and password
- Google sign-in

This improves both usability and realistic application flow.

### 8.2 Product Discovery Module

The product module supports:

- text search
- category filter
- brand-based filtering
- maximum price filter
- minimum rating filter
- sorting by price and rating

This gives users a more practical shopping experience than a simple static product list.

### 8.3 Voice Assistant Module

The assistant supports:

- browser speech recognition
- manual command input
- recorded audio fallback
- spoken assistant replies

If the browser speech service fails, the user can still record a short voice command and send it for backend transcription.

### 8.4 Recommendation Module

The recommendation service provides:

- trending products
- similar products based on category

This improves engagement and makes the assistant feel more like a guided shopping system.

### 8.5 Cart and Checkout Module

The cart system supports:

- add item
- remove item
- read cart
- clear cart at checkout
- total value calculation

Checkout generates a simulated order summary and order ID, allowing the application to demonstrate an end-to-end flow.

## 9. User Interaction Flow

The main runtime flow is:

1. User logs in to the application.
2. User speaks or types a request.
3. Client sends the request either to AI mode or local intent handling.
4. Backend performs the required business operation.
5. Backend returns both:

- assistant response text

- structured UI data such as products, cart, recommendations, or order state

6. Frontend updates the screen immediately.

This architecture is important because it keeps the assistant connected to real UI state instead of behaving like a separate chatbot window.

## 10. Fallback and Reliability Strategy

One of the strongest parts of this project is its fallback design.

### 10.1 Product Fallback

- primary source: FakeStore API
- fallback source: local mock catalog

### 10.2 Cart Fallback

- primary source: Firestore
- fallback source: in-memory storage

### 10.3 Assistant Fallback

- primary mode: OpenAI assistant with tool-calling
- fallback mode: deterministic intent parser on the client

### 10.4 Voice Fallback

- primary mode: browser speech recognition
- fallback mode: recorded audio sent to backend transcription

These fallbacks make the application more suitable for classroom demos and real-world unstable network environments.

## 11. API Summary

The main routes exposed by the backend are:

- ‘GET /api/products‘
- ‘GET /api/products/categories‘
- ‘GET /api/products/recommendations‘
- ‘GET /api/cart‘
- ‘POST /api/cart/add‘
- ‘POST /api/cart/remove‘
- ‘POST /api/cart/checkout‘
- ‘POST /api/ai/chat‘
- ‘POST /api/ai/transcribe‘

This route structure separates product, cart, and AI responsibilities clearly.

## 12. Project Metrics

Based on the current project state:

- source files analyzed: 46
- approximate source lines analyzed: 10,211
- frontend stack: React 18 + Vite 5 + Firebase 11
- backend stack: Node.js + Express 4 + firebase-admin

These numbers show that the project goes beyond a simple prototype and includes multiple integrated modules.

## 13. Challenges Faced

The main implementation challenges include:

- handling browser-specific speech recognition issues
- designing a useful AI assistant that triggers actual shopping actions
- maintaining consistent UI state after AI responses
- ensuring the project still works when external services fail
- balancing a modern interface with a simple demo flow

These challenges required both frontend and backend coordination.

## 14. Limitations

The current version still has some limitations:

- no automated test suite is present yet
- environment example files should be sanitized before final submission
- deployment and production hardening are still limited
- AI service availability depends on valid API quota and billing
- recommendation logic is practical but still basic compared to large-scale commerce systems

## 15. Future Scope

The project can be improved further by adding:

- automated testing for cart and assistant flows
- personalized recommendations based on user history
- multilingual voice support
- order history and analytics dashboard
- better production deployment and monitoring

- richer product sources and category expansion

## 16. Conclusion

Powered Shopping demonstrates how AI, voice interaction, frontend usability, and backend reliability can be combined into a practical domain-specific application. The project is not just a chatbot overlay. It is a functional shopping system that connects authentication, voice input, product discovery, cart management, checkout, and fallback design in a single integrated experience.

The application is suitable for academic demonstration because it clearly shows:

- domain-specific problem solving
- Generative AI API integration
- full-stack system design
- fallback-aware engineering
- a working user-facing prototype

Overall, Powered Shopping serves as a strong example of how modern AI tools can be used to improve user interaction in e-commerce while still respecting usability and system resilience.

---

End of report.